

# Краткое описание пайплайна MNT4 pairing verifier

Итоговая архитектура проекта

22 мая 2026

## 1 Что реализовано

Первая часть — полный on-chain reference для MNT4. Он нужен как контрольная реализация и как экспериментальная демонстрация того, что прямое вычисление MNT4-сопряжения в EVM практически непригодно: актуальный тестовый запуск дает 259 719 954 gas для reference-вычисления digest сопряжения.

Вторая часть — основной вариант работы. Сопряжение считается off-chain в Rust. Solidity-контракт не пересчитывает цикл Миллера и финальную экспоненту. Он получает точки, digest результата, commitments к вспомогательным данным и proof-envelope, затем проверяет, что публичные входы proof согласованы с переданным утверждением.

Общий поток:

```
Rust MNT4 backend
-> MNT4 artifacts and relation roots
-> compact BN254 proof envelope
-> Solidity verifier
```

BN254 используется только как дешевый EVM-совместимый слой проверки proof, потому что Ethereum имеет precompile для BN254 pairing check. Это не заменяет MNT4-вычисление: MNT4-арифметика и MNT4-сопряжение строятся в Rust backend и сверяются с arkworks.

## 2 Off-chain часть

### 2.1 Rust backend

Основной backend находится в:

```
offchain_verify/crates/mnt4_trace_backend
```

Главные файлы:

- `src/main.rs` — CLI-вход: читает request и пишет JSON-артефакты;
- `src/lib.rs` — построение MNT4 artifact, commitments, relation roots и proof input.

Backend принимает точки  $P_i$ , точку  $Q$ , режим fixed-Q или Parametric-Q, context, epoch и служебные поля. В canonical fixture используются  $P$  и  $Q$ , соответствующие генераторам MNT4-753 из arkworks.

### 2.2 Что считается

Основная функция:

```
build_artifact(request) -> PairingArtifact
```

Она выполняет следующие шаги.

1. Нормализует точки  $P_i$  и  $Q$  в формате MNT4-753.
2. Для  $Q$  строит коэффициенты линий цикла Миллера через `arkworks G2Prepared::from(q)`.
3. Упаковывает линии в sparse-формат: отдельно линии удвоения и линии сложения.
4. Считает commitments:

$$C_{dbl} = H(\text{double lines}), \quad C_{add} = H(\text{addition lines}), \quad C_{lines} = H(C_{dbl}, C_{add}).$$

5. Считает Miller loop через `MNT4_753::multi_miller_loop`.
6. Выполняет final exponentiation и получает digest результата сопряжения.
7. Строит relation roots: `lineCacheRelationRoot`, `millerRelationRoot`, `finalExponentiationRelationRoot`.
8. Записывает summary MNT-native relation fragment в поле `nativeRelation`.

## 2.3 Выходные файлы

`write_artifacts` создает:

- `trace.json` — общий artifact;
- `witness.json` — witness-данные;
- `public_inputs.json` — публичные входы;
- `proof_input.json` — вход для генерации proof-envelope.

## 3 MNT-native relation fragment

Отдельный Rust crate:

```
offchain_verify/crates/mnt_cycle_constraints
```

Он нужен для продолжения идеи: MNT4/MNT6 образуют цикл, поэтому отношение для будущего folding можно строить в MNT-sized setting, а не эмулировать MNT4 полностью в BN254.

Текущий compiled relation fragment принимает три публичных root:

- root кэша линий;
- root Miller relation;
- root final exponentiation relation.

Для canonical fixture получено:

- compiled relation constraints: 24 181;
- public inputs: 3;
- witness variables: 72 537;
- relation satisfied: true.

Это pre-folding слой, который показывает порядок стоимости relation-части до оборачивания в folding.

## 4 On-chain контракты

| Контракт                                     | Роль                                                                                                              |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| MNT4PairingFinal                             | Главный контракт. В конструкторе создает proof checker и proof-system verifier. Пользователь вызывает именно его. |
| MNT4PairingVerifier                          | Основная логика проверки: сверяет точки, digest результата, commitments и public inputs proof.                    |
| MNT4PairingProofChecker                      | Нейтральный adapter: проверяет statement hash и передает proof в generated verifier.                              |
| MNT4ProofSystemVerifier                      | Generated verifier, который использует BN254 precompiles Ethereum.                                                |
| MNT4TatePairing,<br>MNT4Extension, BigIntMNT | Арифметика MNT4 для baseline и тестов.                                                                            |

## 5 Главный вызов

Основной метод:

```
verifyPairingClaim(
    G1Point[] pointsP,
    G2Point q,
    bytes32 expectedResultDigest,
    bytes32 artifactRoot,
    bytes32 transcriptHash,
    bytes32 context,
    uint64 epoch,
    ArtifactCommitments commitments,
    bytes proof
) returns (bool)
```

Контракт не доверяет переданному кэшу и не принимает digest результата “на слово”. Он проверяет, что public inputs proof совпадают с утверждением пользователя: hash всех точек  $P_i$ , hash точки  $Q$ , digest результата, artifact root, transcript hash и commitments к line-cache, Miller trace и final exponentiation.

Если пользователь меняет хотя бы один объект, проверка падает. После binding-проверок вызывается:

```
PROOF_CHECKER.verify(statementHash, proof)
```

## 6 Тесты

### 6.1 End-to-end MNT4 verifier

Файл:

```
offchain_verify/test/final_mnt4_pairing/MNT4PairingFinalE2E.t.sol
```

Разворачивает MNT4PairingFinal(8), берет одну точку  $P$  и одну точку  $Q$  из canonical fixture, подает digest результата, commitments и proof-envelope. Ожидаемый результат — true.

В этом же файле есть negative suite: подмена  $P$ , подмена  $Q$ , подмена result digest, подмена line commitment, подмена Miller trace commitment и подмена final exponentiation commitment. Во всех случаях контракт должен отклонить вызов.

## 6.2 Сравнение MNT4 с arkworks

Файл:

```
offchain_verify/test/final_mnt4_pairing/MNT4FinalArkworksCrossCheck.t.sol
```

Проверяется, что Solidity reference-арифметика согласована с canonical vectors из arkworks:

- операции в  $F_p$ ;
- операции в  $F_{p^2}$ ;
- операции в  $F_{p^4}$ ;
- digest MNT4-сопряжения на тех же fixture-точках  $P$  и  $Q$ .

## 6.3 Fuzz-тест boolean-семантики verifier-ов

Тест находится в:

```
offchain_verify/test/final_mnt4_pairing/PairingVerifierBooleanSemanticsFuzz.t.sol
```

Для одного fuzz-сценария accept/reject оба verifier-пути возвращают одинаковый boolean. Корректность MNT4-арифметики на самих MNT4-точках проверяется отдельно через arkworks-векторы.

Foundry генерирует параметр extttuint8 scenario. Если extttscenario четный, выбирается валидный сценарий; если нечетный, выбирается невалидный сценарий.

1. Валидный сценарий. MNT4-путь получает canonical fixture  $P, Q$ , корректный digest результата, commitments и proof-envelope. Он возвращает true. BN254-путь вызывает precompile 0x08 на валидной pairing equation  $e(O, G_2) = 1$ , где  $O = (0, 0)$  — point at infinity в EIP-197 encoding, а  $G_2$  — стандартный BN254  $G_2$  generator. Он тоже возвращает true.
2. Невалидный сценарий. MNT4-путь получает тот же fixture, но первая limb координаты  $x$  точки  $P$  изменяется через  $x[0] \wedge= 1$ ; контракт ловит ошибку binding и результат приводится к false. BN254-путь получает одну пару  $(G_1, G_2)$ , для которой проверка  $e(G_1, G_2) = 1$  ложна; precompile возвращает false.

Используемые BN254 точки:

- $G_1 = (1, 2)$ ;
- $O = (0, 0)$ ;
- $G_2$  в EIP-197 order:  $x\_real=0x198e\dots12c2$ ,  $x\_imag=0x1800\dotsf6ed$ ,  $y\_real=0x0906\dots75b$ ,  $y\_imag=0x12c8\dots7daa$ .

Текущий результат: fuzz-тест проходит на 256 запусках. Gas по последнему запуску: среднее значение — 287 398 gas, медиана — 424 280 gas.

## 6.4 Proof checker tests

Файл:

```
offchain_verify/test/final_mnt4_pairing/MNT4PairingProofChecker.t.sol
```

Проверяется, что валидный proof fixture проходит через checker, statement hash связан с public signal, а подмена public signal отклоняется.

## 6.5 Rust-тесты

Rust backend проверяется отдельно:

- `mnt4_trace_backend`: 15 тестов для line-cache relation, Miller relation, final exponentiation relation, tamper-cases и записи `nativeRelation`;
- `mnt_cycle_constraints`: 10 тестов для operation model, compiled R1CS fragment и negative tamper test для witness root.

## 7 Основные результаты

| Метрика                                                  | Значение              |
|----------------------------------------------------------|-----------------------|
| Полное on-chain reference-вычисление MNT4 pairing digest | 259,719,954 gas       |
| Основной on-chain verifier call                          | 343,435 gas           |
| Backend-only proof checker                               | 325,283 gas           |
| Fuzz boolean semantics, mean                             | 287,398 gas           |
| Fuzz boolean semantics, median                           | 424,280 gas           |
| Rust backend total generation                            | 809 ms                |
| Proof generation                                         | 526 ms                |
| BN254 verifier-envelope                                  | 1,538 constraints     |
| MNT-native accounting model                              | 24,178 constraints    |
| Compiled MNT-native relation fragment                    | 24,181 constraints    |
| Compiled native public inputs                            | 3                     |
| Compiled native witness variables                        | 72,537                |
| BN254 emulated pairing reference                         | 1,393,318 constraints |
| Sonobe-like decider reference                            | 9,000,000 constraints |

## 8 Оценка будущего CycleFold

Сам CycleFold в этой работе не реализован. Реализован слой, который должен стать его входом:

MNT4 artifacts  $\rightarrow$  MNT-native relation fragment  $\rightarrow$  future folding.

Для одной пары текущий compiled relation fragment стоит:

$$C_{rel}(1) = 24\,181 \text{ constraints.}$$

Для multi-pairing используется shared Miller accumulator. По текущей модели:

$$C_{Miller}(1) = 4\,514, \quad C_{Miller}(2) = 7\,520, \quad C_{Miller}(4) = 13\,532.$$

Оценка relation-части без overhead самого folding:

$$C_{rel}(n) \approx C_{lines} + C_{Miller}(n) + C_{FE} + 3,$$

где  $C_{lines} = 19\,554$ ,  $C_{FE} = 110$ .

| Число пар | Relation constraints |
|-----------|----------------------|
| $n = 1$   | 24,181               |
| $n = 2$   | 27,187               |
| $n = 4$   | 33,199               |

Полная стоимость будущего folding-step будет иметь вид:

$$C_{fold-step}(n) = C_{rel}(n) + C_{fold-overhead}.$$

Текущая работа не доказывает готовый CycleFold, но показывает главное предварительное условие: MNT-native relation-часть находится на уровне десятков тысяч constraints. Поэтому следующий этап должен folding-ить именно этот MNT-native слой, а не полный non-native MNT4 pairing circuit внутри BN254.

On-chain стоимость будущей версии при сохранении BN254 compression-envelope ожидается того же порядка, что сейчас:

$$G_{verify} \approx 3.2 \cdot 10^5 - 4.0 \cdot 10^5 \text{ gas},$$

если контракт не пересчитывает MNT4-арифметику on-chain, а проверяет компактный proof.